The Most Efficient Protocol for PAKE:

What Exact Stuff Do You Need to Hash at the End?

Jiayu Xu

Oregon State University, xujiay@oregonstate.edu

1 Introduction

2 Preliminaries

3 Security Analysis of the "Hash Everything" Version

3.1 The Simulator

3.1.1 Simulation Strategy

A large portion of the simulator is routine; below we explain the idea behind the non-trivial parts.

Simulating honest P-to-P' message. When P's session begins, $\mathcal{S}$ must simulate its message (s, T) to P'. This can be easily done: just sample (s, T) at random. However, every time $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$ (let the result be t^*), $\mathcal{S}$ needs to generate a KEM key pair (pk^*, sk^*) for later use (see the "Extracting password guess from malicious P'" paragraph below) and make sure that (pk^*, sk^*) is consistent with the RO queries. This can be done as follows: $\mathcal{S}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*,$$

and programs $H_1(\mathsf{pw}^*, r^*) := R^*$. This also allows $\mathcal{S}$ to store the association between pw^* and (pk^*, sk^*). (Note that (pk^*, sk^*) needs to be freshly generated every time $\mathcal{A}$ queries $H_2(\star, T)$; otherwise R^* never changes and $\mathcal{A}$ can easily find collisions in H_1.)

Extracting password guess from malicious P. When the adversary $\mathcal{A}$ sends a message (s^*, T^*) to P', if it is different from the honest message (s, T), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding

$\mathsf{TestPwd}$ command to $\mathcal{F}_{\mathrm{PAKE}}$. There are two types of attack, which need to be addressed separately:

1. *"Normal" online attack:* $\mathcal{A}$ executes P's algorithm on a password guess pw^*. That is, $\mathcal{A}$ samples public key pk^* and randomness r^*, computes

$$R^* := H_1(\mathsf{pw}^*, r^*), \qquad T^* := pk^* \cdot R^*,$$
$$t^* := H_2(\mathsf{pw}^*, T^*), \qquad s^* := r^* \oplus t^*,$$

   and sends (s^*, T^*) to P'.
   It takes a while to realize how to extract pw^* from (s^*, T^*). First, $\mathcal{S}$ should scan all $H_2(\star, T^*)$ queries, which yields multiple pw^* values (and multiple t^* values); $\mathcal{S}$ must decide which one is the "actual" password guess. This can be done via the following. For every $H_2(\mathsf{pw}^*, T^*)$ query (let the result be t^*), $\mathcal{S}$ computes the corresponding $r^* := s^* \oplus t^*$, and checks if (and when) $H_1(\mathsf{pw}^*, r^*)$ was queried. *With overwhelming probability, there is at most one such H_1 query made before the $H_2(\mathsf{pw}^*, T^*)$ query*, from which $\mathcal{S}$ can extract the "actual" password guess pw^*.

2. *Related key attack:* This attack works only after $\mathcal{A}$ receive an honest (s, T) pair from P. $\mathcal{A}$ skips sampling pk^* or r^*; instead, $\mathcal{A}$ picks any $\Delta \in \mathbb{G}$, computes $T^* := T \cdot \Delta$ and

$$t^* := H_2(\mathsf{pw}^*, T), \qquad (t^*)' := H_2(\mathsf{pw}^*, T^*),$$
$$\delta := t^* \oplus (t')^*, \qquad s^* := s \oplus \delta,$$

   and sends (s^*, T^*) to P'.
   Let us first explain what $\mathcal{A}$ is trying to achieve here. The same randomness r yields two valid messages, (s, T) and (s^*, T^*), such that

$$s \oplus s^* = H_2(\mathsf{pw}, T) \oplus H_2(\mathsf{pw}, T^*),$$

   which allows $\mathcal{A}$ to pick T^* and "reverse engineer" the corresponding s^* if it guesses pw correctly. (s^*, T^*) implicitly encrypts $pk^* = T^*/H_1(\mathsf{pw}, r)$, which $\mathcal{A}$ can compute as $pk \cdot (T^*/T)$.
   To extract pw^* from (s^*, T^*), $\mathcal{S}$ should scan all $H_2(\star, T)$ and $H_2(\star, T^*)$ queries (which yield multiple candidate pw^* values) and check which ones satisfy

$$H_2(\mathsf{pw}^*, T) \oplus H_2(\mathsf{pw}^*, T^*) = s \oplus s^*.$$

   With overwhelming probability, at most one pw^* will satisfy this equation.

Remark: On the term "anchor query". The term "anchor query" was coined in [MRR20, Section 3.3], where it refers to the special $H_1(\mathsf{pw}^*, r^*)$ query made before the $H_2(\mathsf{pw}^*, T^*)$ query in case (1) above; case (2) is completely missing in [MRR20]. This error is fixed in [JRX25], which uses "anchor query" to refer to something else: the $H_2(\mathsf{pw}^*, T^*)$ query in either case (1) or (2) (see [JRX25, Section 4.3.1] and [JRX25, Figure 11, step 4]). This term is not used in [ABJ25].

Extracting password guess from malicious P'. When the adversary $\mathcal{A}$ sends a message (c^*, τ^*) to P, if $\mathcal{A}$ is not passive (i.e., it is not the case that $(s^*, T^*) = (s, T)$ *and* $(c^*, \tau^*) = (c, \tau)$), the simulator $\mathcal{S}$ must extract a password guess (if any) and send a corresponding $\mathsf{TestPwd}$ command to $\mathcal{F}_{\mathrm{PAKE}}$.

$\mathcal{S}$ should find the H_{tag} query whose result is τ^*, which includes $(\mathsf{pw}^*, pk^*, K^*)$. However, pw^* constitutes a password guess only if it is consistent with pk^* and K^*, i.e., $K^* = \mathsf{Decap}(sk^*, c^*)$ for sk^* corresponding to pk^*. $\mathcal{S}$ can check this condition because it has stored the correspondence between pw^* and (pk^*, sk^*) (see the "Simulating honest P-to-P' message" paragraph above), and can use sk^* to check if the equation holds. (This extraction strategy does not work anymore if *any* of pw^*, pk^*, c^* is not included in H_{tag}. In later sections we will explain how to simulate other versions of the protocol.)

3.1.2 Formal Description of the Simulator

3.2 Hybrid Argument

We now prove that for any PPT environment $\mathcal{Z}$, the simulator $\mathcal{S}$ in Section 3.1.2 generates an ideal-world view indistinguishable from $\mathcal{Z}$'s real-world view. The sequence of games start from the real world (Game 0) and ends at the ideal world.

Game 0: This is the real world. Recall that the passwords of P and P' are pw and pw', respectively; the flow of events is:

- P sends (s, T) to P';
- It is intercepted by the man-in-the-middle adversary $\mathcal{A}$, who sends (s^*, T^*) (which may or may not be equal to (s, T)) to P';
- P' outputs session key SK' and sends (c, τ') to P;
- It is again intercepted by $\mathcal{A}$, who sends (c^*, τ^*) to P;
- P computes τ. If $\tau^* = \tau$ then P outputs SK, otherwise P outputs $\perp$.

(Note that there are three τ values: τ' computed and sent by P', τ^* sent by $\mathcal{A}$, and τ computed by P used in P's check.)

Passwords match and $\mathcal{A}$ is passive, except maybe modifying the tag.

Game 1: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, do the following:

- If $\tau^* = \tau'$ (i.e., $\mathcal{A}$ is passive), set $SK := SK'$.
- If $\tau^* \neq \tau'$, set $SK := \perp$.

By correctness of the protocol (see Section 2), in Game 0 $\tau = \tau'$ except with probability $\mathbf{Adv}^{\text{correctness}}$. This means that except with probability $\mathbf{Adv}^{\text{correctness}}$, P's check $\tau^* \stackrel{?}{=} \tau$ in Game 0 is equivalent to P's check $\tau^* \stackrel{?}{=} \tau'$ in Game 1. If the check passes then both games set $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c, K)$, otherwise both games set $SK := \perp$. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{0,1} = \mathbf{Adv}^{\text{correctness}}.$$

This game corresponds to games G_3–G_5 in [ABJ25]. We believe having a single game that handles both the $\tau^* = \tau'$ case and the $\tau^* \neq \tau'$ case is much cleaner.

P and P''s passwords match, and $\mathcal{A}$ forwards the P-to-P' message.

Game 2: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query resulting in τ^*. If there is none or more than one, set $SK := \perp$. If there is a unique such query, do the following:

- If $K^* = K$, set $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c^*, K)$ (i.e., there is no change from Game 1).
- If $K^* \neq K$, set $SK := \perp$.

Claim 1. *The probability that P' queries $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is at most $(q_1 + 1)/|\mathbb{G}|$.*

P' makes a single H_{tag} query, on $(\mathsf{pw}', pk', c, K')$. If $\mathsf{pw} \neq \mathsf{pw}' \vee c^* \neq c$ then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$. Otherwise $\mathsf{pw} = \mathsf{pw}' \wedge c^* = c$, so $(s^*, T^*) \neq (s, T)$. We upper-bound $\Pr[pk' = pk]$ in this case:

We have

$$pk = \frac{T}{R} = \frac{T}{H_1(\mathsf{pw}, r)} = \frac{T}{H_1(\mathsf{pw}, s \oplus t)} = \frac{T}{H_1(\mathsf{pw}, s \oplus H_2(\mathsf{pw}, T))},$$
$$pk' = \frac{T^*}{R'} = \frac{T^*}{H_1(\mathsf{pw}, r')} = \frac{T^*}{H_1(\mathsf{pw}, s^* \oplus t')} = \frac{T^*}{H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))}.$$

There are two cases:

- $s \oplus H_2(\mathsf{pw}, T) = s^* \oplus H_2(\mathsf{pw}, T^*)$. In this case $T^* \neq T$ (otherwise $T^* = T \Rightarrow H_2(\mathsf{pw}, T^*) = H_2(\mathsf{pw}, T) \Rightarrow s^* = s$, so $(s^*, T^*) = (s, T)$, which is impossible). But then it is impossible that $pk' = pk$.
- $s \oplus H_2(\mathsf{pw}, T) \neq s^* \oplus H_2(\mathsf{pw}, T^*)$. In this case $H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))$ is uniformly random in $\mathbb{G}$ and independent of both pk and T^*, so the probability that $pk' = pk$, i.e., $H_1(\mathsf{pw}, s^* \oplus H_2(\mathsf{pw}, T^*))$ happens to be equal to T^*/pk, is $1/|\mathbb{G}|$ for a single H_1 query.

Since there are $q_1 + 1$ H_1 queries (q_1 by $\mathcal{A}$ and one by P'), the probability that $pk' = pk$ is upper-bounded by $(q_1 + 1)/|\mathbb{G}|$. If $pk' \neq pk$ then of course P''s query is not $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$. This yields the claim.

Going back to the game hop, Game 1 and Game 2 are identical unless of the following happens:

1. $\mathcal{A}$ never makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^*, yet P's check $\tau^* \stackrel{?}{=} H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ passes.

2. $\mathcal{A}$ makes two different $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ queries, both resulting in τ^*.

3. $\mathcal{A}$ makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query resulting in τ^*, and P's $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ query also results in τ^*.

In the first event, by the claim P' does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ except with probability up to $(q_1+1)/|\mathbb{G}|$. $\mathcal{A}$ itself also does not query $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ (otherwise the result is τ^*). So $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is independent of $\mathcal{A}$'s view, and the probability that $\mathcal{A}$ sends $\tau^* = H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K)$ is $1/2^{2n}$. The second and third events are collision in H_{tag}, whose (combined) probability is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}}+1)/2^{2n+1}$. Overall,

$$\mathbf{Dist}_{\mathcal{Z}}^{1,2} \leq \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}}+1)+2}{2^{2n+1}} + \frac{q_1+1}{|\mathbb{G}|}.$$

This game corresponds to games G_6 and G_7 in [ABJ25]. However, there are two differences:

1. *The protocol in [ABJ25] additionally includes (s, T) as part of P's input to H_{tag}. Then Claim 1 is trivial (and the probability in the claim is 0): P's H_{tag} query is on $(\mathsf{pw}, pk, s, T, c^*, K)$ and P''s H_{tag} query is on $(\mathsf{pw}', pk', s^*, T^*, c, K')$; if they are equal then $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, violating the assumption of this game.*

2. *The H_{tag} collision cases (events (2) and (3) above) were ruled out in game G_2 in [ABJ25] (as aborting condition [A1]).*

The proof of Claim 1 critically relies on pw, pk, c being included in H_{tag} inputs: pw and c have to be included in order to rule out the $\mathsf{pw} \neq \mathsf{pw}' \vee c^* \neq c$ case; pk has to be included because in the $\mathsf{pw} = \mathsf{pw}' \wedge c^* = c$ case, we show that P and P''s H_{tag} queries are different by comparing P's query input pk and P''s query input pk'.

Game 3: In the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0,1\}^n$. ($c$ is still computed as before.)

Let Query_1 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(\mathsf{pw}, pk, c, K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, K')$. Clearly Game 2 and Game 3 are identical unless Query_1 happens. We upper-bound $\Pr[\mathsf{Query}_1]$ via a reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property of KEM:

Reduction $\mathcal{R}_{\text{OW-1PCA1}}$: // $\boxed{\text{Boxed text}}$ indicates the single PC oracle query; same for reductions below

Sample $i \leftarrow [q_{\mathsf{key}} + q_{\mathsf{tag}}]$ as a guess that $\mathcal{A}$'s i-th H_{key} or H_{tag} query triggers Query_1.
On input (pk, c),

1. Invoke $\mathcal{Z}$. On ($\mathsf{NewSession}, sid, P, P', \mathsf{pw}$, "initiator") from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk rather than $(pk, \star) \leftarrow \mathsf{Gen}$. (Note that computing (s, T) does not require knowing sk.) Send (s, T) to $\mathcal{A}$.

2. On ($\mathsf{NewSession}, sid, P', P, \mathsf{pw}'$, "respondent") from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{OW-PCA1}}$'s inputs.)

3. On (c^*, τ^*) from $\mathcal{A}$,

   (a) If $c^* = c$, do what's described in Game 1.

   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

   (c) $\boxed{\text{Query } \mathsf{PC}(sk, c^*, K^*)}$. // This is a valid query since we do this step only if $c^* \neq c$
   If the result is 1, output $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c^*, K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. On $\mathcal{A}$'s i-th H_{key} or H_{tag} query, if it is in the form of $H_{\mathsf{key}}(\mathsf{pw}, pk, c, (K')^*)$ or $H_{\mathsf{tag}}(\mathsf{pw}, pk, c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_1 happens, the only difference between Game 2/3 and $\mathcal{R}_{\text{OW-1PCA1}}$'s simulation (until Query_1 happens) is that $\mathcal{R}_{\text{OW-1PCA1}}$ uses its input pk on the P side, and then uses its input $c \leftarrow \mathsf{Encap}(pk)$ on the P' side. Since we assume $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, in Game 2/3 P''s algorithm will recover $pk' = pk$, so there c is also computed as $\mathsf{Encap}(pk)$ and thus $\mathcal{R}_{\text{OW-1PCA1}}$ simulates Game 2/3 perfectly. Furthermore, if $\mathcal{R}_{\text{OW-1PCA1}}$'s guess is correct then it wins the OW-1PCA game. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{2,3} \leq \Pr[\mathsf{Query}_1] \leq (q_{\mathsf{key}} + q_{\mathsf{tag}})\mathbf{Adv}_{\mathcal{R}_{\text{OW-1PCA1}}}^{\text{OW-1PCA}}.$$

This game corresponds to game G_8 in [ABJ25]. However, there are two differences:

1. *[ABJ25] reduces to OW-PCA (where the KEM adversary can query its plaintext-checking oracle polynomially many times), rather than OW-1PCA. In this case, step 4 of the reduction above can query $\mathsf{PC}(sk, c, (K')^*)$ $q_{\mathsf{key}} + q_{\mathsf{tag}}$ times to check which of $\mathcal{A}$'s query (if any) triggers Query_1, hence avoiding the $q_{\mathsf{key}} + q_{\mathsf{tag}}$ loss in its advantage.*

2. *The above also means that the plaintext-checking oracle in their OW-PCA game has to allow for $\mathsf{PC}(sk, c, \star)$ queries, whereas our reduction to OW-1PCA never makes a query in this form and thus the plaintext-checking oracle in our OW-1PCA game can disallow such queries.*

Game 4: Again in the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, generate P''s KEM public key $(pk', \star) \leftarrow \mathsf{Gen}$ and compute $(c, \star) \leftarrow \mathsf{Encap}(pk')$. (Note that K' is not used anywhere since Game 3 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.)

The difference between Game 3 and Game 4 is that in Game 3 both P and P' use the same pk, whereas in Game 4 P uses pk and P' uses an independent pk'. We construct a reduction $\mathcal{R}_{\text{ANO-1PCA}}$ to the ANO-1PCA property of KEM. This reduction, in fact, is identical to the reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property in Game 3 except the last step; for completeness, we still present the full reduction and highlight the difference in blue:

Reduction $\mathcal{R}_{\text{ANO-1PCA}}$:

On input (pk, c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{“initiator”})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T), except that $\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk rather than $(pk, \star) \leftarrow \mathsf{Gen}$. (Note that computing (s, T) does not require knowing sk.) Send (s, T) to $\mathcal{A}$.

2. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{ANO-PCA1}}$'s inputs.)

3. On (c^*, τ^*) from $\mathcal{A}$,

   (a) If $c^* = c$, do what's described in Game 1.

   (b) If $c^* \neq c$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

   (c) $\boxed{\text{Query } \mathsf{PC}(sk, c^*, K^*)}$. // This is a valid query since we do this step only if $c^* \neq c$
   If the result is 1, output $SK := H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ to $\mathcal{Z}$.
   If the result is 0, output $SK := \perp$ to $\mathcal{Z}$.

4. When $\mathcal{Z}$ outputs bit b^*, copy $\mathcal{Z}$'s output.

$\mathcal{R}_{\text{ANO-1PCA}}$ uses its input pk on the P side and c on the P' side. If $b = 0$ then $c \leftarrow \mathsf{Encap}(pk)$, i.e., both P and P' use the same pk, so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 3; if $b = 1$ then $c \leftarrow \mathsf{Encap}(pk')$, i.e., P' uses an independently generated pk', so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 4. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{3,4} \leq \mathbf{Adv}_{\mathcal{R}_{\text{ANO-1PCA}}}^{\text{ANO-1PCA}}.$$

This game corresponds to game G_9 in [ABJ25]. However, there are two differences:

1. *[ABJ25] reduces to ANO-PCA (where the KEM adversary can query its plaintext-checking oracle polynomially many times), rather than ANO-1PCA. Unlike in Game 3, here reducing to this stronger assumption does not provide any advantage.*

2. *The assumption [ABJ25] reduces to is even further stronger in that their KEM adversary's inputs are (pk, pk', c), whereas we do not give pk' to the KEM adversary. Note that the reduction does not need pk' at all (it only needs to send c which is an encapsulation of either pk or pk'), so the assumption in [ABJ25] is unnecessarily strong.*

P and P''s passwords do not match, and $\mathcal{A}$ forwards the P-to-P' message.

Game 5: When $\mathcal{A}$ makes a fresh query $H_2(\mathsf{pw}^*, T)$ (let the result be t^*), generate $(pk^*, sk^*) \leftarrow \mathsf{Gen}$, and compute

$$r^* := s \oplus t^*, \quad R^* := T/pk^*.$$

((s, T) are the honest P-to-P' message.) If $H_1(\mathsf{pw}^*, r^*)$ has already been defined and it is not R^*, abort. Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$, and store $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$.

Furthermore, in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, when P' queries $H_2(\mathsf{pw}', T)$, use the method described above to generate pk', r', R' and program H_1. (Note that the case where $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ has already been addressed in Game 3 and Game 4, and P' does not need to make an H_2 query there.)[1]

In the analysis below, we combine $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T)$ queries and P''s (possible) $H_2(\mathsf{pw}', T)$ query, and call all of them "$H_2(\mathsf{pw}^*, T)$". We first upper-bound the aborting probability. For every fresh $H_2(\mathsf{pw}^*, T)$ query, an independent $t^* \leftarrow \{0,1\}^{3n}$ value is sampled, which results in an independent $r^* \leftarrow \{0,1\}^{3n}$ (regardless of the distribution of s). As such, there are up to $q_2 + 1$ independent and uniformly random r^* values. The probability that one of $\mathcal{A}$'s q_1 $H_1(\mathsf{pw}^*, r^*)$ query inputs happens to be any of them is upper-bounded by $q_1(q_2+1)/2^{3n}$.

Now suppose the aborting condition does not happen. The difference between Game 4 and Game 5 is that Game 4 samples $H_1(\mathsf{pw}^*, r^*)$ uniformly at random, whereas Game 5 defines it as T/pk^*. Note that pk^* is not used anywhere else in the game, so the distinguishing advantage between Game 4 and Game 5 can be reduced to the PR-PK property of KEM. We only sketch the reduction $\mathcal{R}_{\text{PR-PK}}$ here: $\mathcal{R}_{\text{PR-PK}}$, on input pk^* (which is either an honestly generated public key or a uniformly random value), samples $i \leftarrow [q_2 + 1]$, and

[1] A slightly confusing case is when $s^* \neq s \wedge T^* = T$, causing P' to query $t' := H_2(\mathsf{pw}', T)$; and $\mathcal{A}$ also queries $H_2(\mathsf{pw}', T)$. In this case P''s $r' = s^* \oplus t'$ and $\mathcal{A}$'s $r^* = s \oplus t'$ are different, causing P''s values R', pk' and $\mathcal{A}$'s values R^*, pk^* to be independent of each other. Therefore, the game should run P''s algorithm as usual when P' queries $H_2(\mathsf{pw}', T)$ (which generates R', pk'), and use the new method described in Game 5 when $\mathcal{A}$ makes the same query (which generates R^*, pk^*).

answers the (either $\mathcal{A}$'s or an honest parties') first $i-1$ H_2 queries as in Game 4 and the last $q_2 - i + 1$ H_2 queries as in Game 5; for the i-th H_2 query, $\mathcal{R}_{\text{PR-PK}}$ computes

$$r^* := s \oplus t^*, \quad R^* := T/pk^*$$

and programs $H_1(\mathsf{pw}^*, r^*) := R^*$. (Note that the $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$ record is not actually used in Game 5, so $\mathcal{R}_{\text{PR-PK}}$ does not need to store it.) $\mathcal{R}_{\text{PR-PK}}$ simulates other parts of Game 4/5 honestly. Finally, $\mathcal{R}_{\text{PR-PK}}$ copies $\mathcal{Z}$'s output bit.

A standard hybrid argument shows that (assuming the aborting condition does not happen) $\mathcal{Z}$'s distinguishing advantage between Game 4 and Game 5 is upper-bounded by $(q_2+1)\mathbf{Adv}^{\text{PR-PK}}_{\mathcal{R}_{\text{PR-PK}}}$.[2] Overall, we have

$$\mathbf{Dist}^{4,5}_{\mathcal{Z}} \leq (q_2+1)\mathbf{Adv}^{\text{PR-PK}}_{\mathcal{R}_{\text{PR-PK}}} + \frac{q_1(q_2+1)}{2^{3n}}.$$

This game corresponds to game G_{10} in [ABJ25]. However, there are two differences:

1. *[ABJ25] runs the procedure above to generate pk', r', R' and program H_1 as long as P' makes an $H_2(\mathsf{pw}', T)$ query — that is, including the case where $s^* \neq s \wedge T^* = T$. (If $T^* \neq T$ then of course P' does not make an $H_2(\mathsf{pw}', T)$ query and there is no change from the real world.) The reason for making this change is to prepare for Game 6 and Game 7 below, which simplifies P''s algorithm when $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$ (i.e., if we did not make the change in Game 5 first, the reductions in Game 6 and Game 7 would not go through); therefore, it is unnecessary (and potentially confusing) to make this change when $s^* \neq s \wedge T^* = T$.*

2. *The aborting case was ruled out in game G_2 in [ABJ25] (as aborting condition [A2b1]).*

Game 6: In the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, sample $SK', \tau' \leftarrow \{0,1\}^n$.

Let Query_2 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(\mathsf{pw}', pk', c, K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}', pk', c, K')$. Clearly Game 5 and Game 6 are identical unless Query_2 happens. We upper-bound $\Pr[\mathsf{Query}_2]$ via a reduction $\mathcal{R}_{\text{OW1}}$ to the OW property of KEM. This is somewhat similar to the reduction $\mathcal{R}_{\text{OW-1PCA1}}$ to the OW-1PCA property in Game 3, and we highlight the differences in blue:

Reduction $\mathcal{R}_{\text{OW1}}$:

Sample $i \leftarrow [q_{\mathsf{key}} + q_{\mathsf{tag}}]$ as a guess that $\mathcal{A}$'s i-th H_{key} or H_{tag} query triggers Query_2, and $j \leftarrow [q_2+1]$ as a guess that the j-th H_2 query (by either $\mathcal{A}$ or P') is $H_2(\mathsf{pw}', T)$.
On input (pk', c),

[2] Note that here we cannot make q_2+1 separate reductions to the PR-PK property, and must reduce to PR-PK "in one blow". Furthermore, the calculation of the reduction's advantage is more complicated than the normal case where the reduction wins as long as its guess is correct. This is similar to, say, why composing a PRG polynomial times yields another PRG; see, e.g., [BS23, Section 3.4.3] for details.

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{"initiator"})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T). Send (s, T) to $\mathcal{A}$.

2. Whenever there is an $H_2(\star, T)$ query (by either $\mathcal{A}$ or $\mathcal{R}_{\mathrm{OW1}}$ itself on behalf of P' — see step 3 below), do what's described in Game 5. However, if this is the j-th query, use $\mathcal{R}_{\mathrm{OW1}}$'s input pk' instead of generating pk^* freshly.

3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{"respondent"})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise

   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T)$, make this query on behalf of P'.

   (b) Sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\mathrm{OW1}}$'s inputs.)

4. On (c^*, τ^*) from $\mathcal{A}$,

   (a) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)

   (b) Compute $K := \mathsf{Decap}(sk, c)$. ($sk$ is the KEM secret key generated in step 1.) // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
   If $K = K^*$, output $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c^*, K)$ to $\mathcal{Z}$.
   If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.

5. On $\mathcal{A}$'s i-th H_{key} or H_{tag} query, if it is in the form of $H_{\mathsf{key}}(\mathsf{pw}', pk', c, (K')^*)$ or $H_{\mathsf{tag}}(\mathsf{pw}', pk', c, (K')^*)$ for some $(K')^*$, output $(K')^*$; otherwise abort.

Assuming Query_2 happens and $\mathcal{R}_{\mathrm{OW1}}$'s guess of index j is correct, the only difference between Game 5/6 and $\mathcal{R}_{\mathrm{OW1}}$'s simulation (until Query_2 happens) is that $\mathcal{R}_{\mathrm{OW1}}$ uses its input pk' to program $H_1(\mathsf{pw}', r')$, and then uses its input $c \leftarrow \mathsf{Encap}(pk')$ as the P'-to-P message. Since c is also computed as $\mathsf{Encap}(pk')$ in Game 5/6, $\mathcal{R}_{\mathrm{OW1}}$ simulates Game 5/6 perfectly. Furthermore, if $\mathcal{R}_{\mathrm{OW1}}$'s guess of index i is correct then it wins the OW game. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{5,6} \leq \Pr[\mathsf{Query}_2] \leq (q_2 + 1)(q_{\mathsf{key}} + q_{\mathsf{tag}})\mathbf{Adv}_{\mathcal{R}_{\mathrm{OW1}}}^{\mathrm{OW}}.$$

Remark. It is very subtle why the reduction needs to guess the index j and lose a factor of $q_2 + 1$ (and we applaud [ABJ25] for getting it right). The reason is as follows. Consider the following environment $\mathcal{Z}$:

1. Initiate P's session on password pw and receive (s, T).

2. Instruct $\mathcal{A}$ to make a large number of $H_2(\mathsf{pw}^*, T)$ queries (let the result be t^*), one of which is $H_2(\mathsf{pw}', T)$ for a special value pw'.

3. For each of the H_2 queries in step 2,

$$\text{compute } r^* := s \oplus t^*, \quad \text{query } H_1(\mathsf{pw}^*, r^*).$$

4. Initiate P''s session on password pw' and instruct $\mathcal{A}$ to forward (s, T) to P'.

To simulate Game 5/6 correctly, $\mathcal{R}_{\mathrm{OW1}}$ must use its input pk' to program $H_1(\mathsf{pw}', r') := T/pk'$ in step 3. The problem is that $\mathcal{R}_{\mathrm{OW1}}$ *does not know* pw' *until step 4, so it has no idea which* $H_1(\mathsf{pw}', r')$ *query in step 3 needs to be programmed using* pk'! The only way to get around is to make a guess over all H_2 queries, which incurs a $q_2 + 1$ loss. (If we require $\mathcal{Z}$ to always initiate P and P''s sessions simultaneously, this loss can be avoided since in step 3 $\mathcal{R}_{\mathrm{OW1}}$ must have already learned pw'.)

This game corresponds to game G_{11} *in [ABJ25], except that they reduce to OW-PCA and avoids the* $q_{\mathsf{key}} + q_{\mathsf{tag}}$ *loss (cf. the comment after Game 3).*

Game 7: Again in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, recall Game 6 does the following on the P' side: Generate $(pk', \star) \leftarrow \mathsf{Gen}$, query $t' := H_2(\mathsf{pw}', T)$, and compute

$$r' := s \oplus t', \quad R' := T/pk'.$$

((s, T) are the honest P-to-P' message.) If $H_1(\mathsf{pw}', r')$ has already been defined and it is not R', abort. Otherwise program $H_1(\mathsf{pw}', r') := R'$, $\underline{\text{compute } (c, \star) \leftarrow \mathsf{Encap}(pk')}$, and sample $SK', \tau' \leftarrow \{0,1\}^n$. Finally, output SK' and send (c, τ') to P. (Note that K' is not used anywhere since Game 6 — in particular, P' does not use K' to compute SK' or τ' anymore — so we do not need to explicitly mention it.) In this game, we independently generate $(pk'', \star) \leftarrow \mathsf{Gen}$ and compute $(c, \star) \leftarrow \mathsf{Encap}(pk'')$ (the underlined expression).

The difference between Game 6 and Game 7 is that in Game 6 c is generated by

$$pk' = \frac{T}{H_1(\mathsf{pw}', s \oplus H_2(\mathsf{pw}', T))}$$

which $\mathcal{A}$ can compute, whereas in Game 7 c is generated by an independent pk''. We construct a reduction $\mathcal{R}_{\mathrm{ANO1}}$ to the ANO property of KEM. This reduction, in fact, is identical to the reduction $\mathcal{R}_{\mathrm{OW1}}$ to the OW property in Game 6 except the last step; for completeness, we still present the full reduction and highlight the difference in blue:

Reduction $\mathcal{R}_{\mathrm{ANO1}}$:

Sample $j \leftarrow [q_2 + 1]$ as a guess that the j-th H_2 query (by either $\mathcal{A}$ or P') is $H_2(\mathsf{pw}', T)$.
On input (pk', c),

1. Invoke $\mathcal{Z}$. On $(\mathsf{NewSession}, sid, P, P', \mathsf{pw}, \text{"initiator"})$ from $\mathcal{Z}$, run P's algorithm to compute (s, T). Send (s, T) to $\mathcal{A}$.

2. Whenever there is an $H_2(\star, T)$ query (by either $\mathcal{A}$ or $\mathcal{R}_{\mathrm{ANO1}}$ itself on behalf of P' — see step 3 below), do what's described in Game 5. However, if this is the j-th query, use $\mathcal{R}_{\mathrm{ANO1}}$'s input pk' instead of generating pk^* freshly.

3. On $(\mathsf{NewSession}, sid, P', P, \mathsf{pw}', \text{“respondent”})$ from $\mathcal{Z}$ and (s^*, T^*) from $\mathcal{A}$, if $\neg(\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T))$, abort. Otherwise
   (a) If $\mathcal{A}$ has not queried $H_2(\mathsf{pw}', T)$, make this query on behalf of P'.
   (b) Sample $SK', \tau' \leftarrow \{0,1\}^n$, output SK' to $\mathcal{Z}$, and send (c, τ') to $\mathcal{A}$. (c is part of $\mathcal{R}_{\text{ANO1}}$'s inputs.)
4. On (c^*, τ^*) from $\mathcal{A}$,
   (a) Find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query whose result is τ^*. If there is none or more than one, output $SK := \perp$ to $\mathcal{Z}$. (This matches Game 2.)
   (b) Compute $K := \mathsf{Decap}(sk, c)$. ($sk$ is the KEM secret key generated in step 1.) // No need to query a plaintext-checking oracle since pk on the P side and pk' on the P' side are independent of each other, so the reduction can sample sk on its own
   If $K = K^*$, output $SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c^*, K)$ to $\mathcal{Z}$.
   If $K \neq K^*$, output $SK := \perp$ to $\mathcal{Z}$.
5. When $\mathcal{Z}$ outputs bit b^*, copy $\mathcal{Z}$'s output.

Assume $\mathcal{R}_{\text{ANO1}}$'s guess of index j is correct. If $b = 0$ then $c \leftarrow \mathsf{Encap}(pk')$, i.e., the computation of R' and c use the same pk', so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 6; if $b = 1$ then $c \leftarrow \mathsf{Encap}(pk'')$, i.e., the computation of c uses an independently generated pk'', so $\mathcal{R}_{\text{ANO-1PCA}}$ simulates Game 7. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{6,7} \leq (q_2 + 1)\mathbf{Adv}_{\mathcal{R}_{\text{ANO-1PCA}}}^{\text{ANO-1PCA}}.$$

Game 8: Again in the case where $\mathsf{pw} \neq \mathsf{pw}' \wedge (s^*, T^*) = (s, T)$, we outright skip the computation of t', r', R' and the programming of H_1 (the brown text in Game 6). (Since pk' is not generated anymore, we can rename pk'' — the KEM public key from which c is computed — to pk'.)

Note that the aborting condition, which was originally introduced in Game 5, is removed (for the specific $H_2(\mathsf{pw}', T)$ query by P'). By an analysis similar to that in Game 5, the aborting probability in Game 7 is upper-bounded by $q_1/2^{3n}$. If the aborting condition does not happen, the brown text is independent of the rest of the game, so removing it does not affect the distribution of $\mathcal{Z}$'s output. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{6,7} \leq \frac{q_1}{2^{3n}}.$$

The two games above combined correspond to game G_{12} in [ABJ25]. [ABJ25] does not have the intermediate Game 7 in our proof, and outright removes the brown text from what's our Game 6. We believe it is clearer to include our Game 7, where H_2 is still queried and H_1 is still programmed: in this way it is easier to see why the reduction to ANO simulates the games correctly. (Of course, it is possible that an experienced cryptographer can see through the two game hops in one shot, but the author of this paper admits that he is not that experienced and *has to work out the game hops and reductions the "dumb" way.) Also, [ABJ25] reduces to ANO-PCA*[3]*, although it also says that the plaintext-checking oracle is not needed.*

Computation of the P-to-P' message.

Game 9: Note that up to now we have not changed P's algorithm before sending (s, T) (in particular, Game 5 dealt with H_2 queries by $\mathcal{A}$ and P', but not P). In this game we modify it as follows: Sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$. If $\mathcal{A}$ has queried $H_2(\star, T)$, abort. Otherwise query $t := H_2(\mathsf{pw}, T)$, generate $(pk, sk) \leftarrow \mathsf{Gen}$, and compute

$$r := s \oplus t, \quad R := T/pk.$$

If $H_1(\mathsf{pw}, r)$ has already been defined and it is not R, abort. Otherwise program $H_1(\mathsf{pw}, r) := R$.

We first upper-bound the aborting probability. Apart from the one made by P, there are q_2 H_2 queries in total (note that P' does not query H_2 anymore since Game 8). Thus, the probability that one of the query inputs happens to contain T is upper-bounded by $q_2/|\mathbb{G}|$. By an analysis similar to that in Game 5 and Game 8, the probability of the second aborting event is upper-bounded by $q_1/2^{3n}$.

Now suppose neither of the aborting events happens. The following equations hold in both Game 8 and Game 9:

$$r = s \oplus H_2(\mathsf{pw}, T), \quad T = R \cdot pk.$$

Here, pk is generated by Gen, and $H_2(\mathsf{pw}, T)$ is fixed by an H_2 query once T is fixed. The four remaining variables r, s, R, T are constrained by two equations and have $4 - 2 = 2$ degrees of freedom: if we sample $r \leftarrow \{0,1\}^{3n}, R \leftarrow \mathbb{G}$ then we get Game 8, and if we sample $s \leftarrow \{0,1\}^{3n}, T \leftarrow \mathbb{G}$ then we get Game 9. Thus, Game 8 and Game 9 are identical. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{8,9} \le \frac{q_1}{2^{3n}} + \frac{q_2}{|\mathbb{G}|}.$$

The game above corresponds to game G_{13} in [ABJ25], with one difference: our game aborts if $\mathcal{A}$ has queried $H_2(\mathsf{pw}^*, T)$ for any pw^* when T is sampled, whereas the game in [ABJ25] aborts only if $\mathcal{A}$ has queried $H_2(\mathsf{pw}, T)$. Note that the simulator does not know pw, so it cannot tell if $\mathcal{A}$ has queried $H_2(\mathsf{pw}, T)$ and instead has to abort as long as $\mathcal{A}$ has queried $H_2(\star, T)$. (The simulator in [ABJ25] is correct, but its game hops do not match the ideal world regarding which H_2 queries will trigger an abortion.) Also, we believe our argument under this game is much more concise and cleaner (our presentation is, in spirit, similar to [JRX25, Lemma 4.5, Hybrid 2]).

Game 10: When P's session begins, halt after checking if $H_2(\star, T)$ has been defined (i.e., before the brown text in Game 9). Then when P receives (c^*, τ^*),

[3] [ABJ25] actually mentions ANO-CPA, which appears to be a typo.

if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, do what's described in Game 1. Otherwise,

- If $H_2(\mathsf{pw}, T)$ has now been defined (via a query by $\mathcal{A}$), there must be a corresponding record $\langle \mathsf{pw}, \star, sk \rangle$ (see Game 5).
- If $H_2(\mathsf{pw}, T)$ is still undefined, run the brown text in Game 9.

Either way, an sk is defined and the game can proceed as usual.

The only difference between Game 9 and Game 10 lies in the fact that when we move the brown text around, the game might abort — as part of the brown text — at different times. Since the aborting event happens with probability at most $q_1/2^{3n}$, we have

$$\mathbf{Dist}_{\mathcal{Z}}^{9,10} \leq \frac{q_1}{2^{3n}}.$$

The game above corresponds to game G_{14} in [ABJ25]. Note that we have a $q_1/2^{3n}$ loss in both Game 9 and Game 10; if we remove Game 9 and directly jump to Game 10 from Game 8, this loss does not need to be repeated (since Game 8 and Game 10 are identical unless one of the two aborting events happens). We (as well as [ABJ25]) choose to include Game 9 for clarity.

Intermission. Let's pause a bit and review the changes we have made so far:

1. *P-to-P' message:* Game 10 outright samples (s, T) at random. [changed in Game 9]
2. *P''s session key and P'-to-P message:* On (s^*, T^*) from $\mathcal{A}$ to P', if $(s^*, T^*) = (s, T)$, Game 10 outright samples SK', τ' at random. Regarding c, Game 10 generates $(pk', \star) \leftarrow \mathsf{Gen}$ and computes $c \leftarrow \mathsf{Encap}(pk')$. (If $(s^*, T^*) \neq (s, T)$ then there is no change from the real world.) [changed in Games 3, 4, 6, 7, 8]
3. *P's session key:* On (c^*, τ^*) from $\mathcal{A}$ to P, we have made two changes:
   (a) If $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$, Game 10 outright sets $SK := SK'$ if $\tau^* = \tau'$ and $SK := \perp$ otherwise. [changed in Game 1]
   (b) Otherwise if $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ resulting in τ^*, or more than one such query, Game 10 outright sets $SK := \perp$. [changed in Game 2]
   (c) Otherwise if sk has not been defined yet, Game 10 queries $H_2(\mathsf{pw}, T)$ on behalf of P, which generates sk (see below). The remaining part is unchanged from the real world.
4. *H_2 queries:* When either $\mathcal{A}$ or P makes a fresh query $H_2(\mathsf{pw}^*, T)$ (let the result be t^*), Game 10 runs the following procedure: Generate $(pk^*, sk^*) \leftarrow \mathsf{Gen}$, and compute

$$r^* := s \oplus t^*, \quad R^* := T/pk^*.$$

If $H_1(\mathsf{pw}^*, r^*)$ has already been defined and it is not R^*, abort. Otherwise program $H_1(\mathsf{pw}^*, r^*) := R^*$. [changed in Games 5, 10]

See Figure 1 for a summary of Game 10.

Password extraction from P'-to-P message.

Game 11: In the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, if there is no $\langle \mathsf{pw}, sk \rangle$ record, set $SK := \perp$ (i.e., replace the brown text in Figure 1 with "$SK := \perp$").

Game 10 and Game 11 differ only if $\mathcal{A}$ does not make an $H_2(\mathsf{pw}, T)$ query, but makes an $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*)$ query resulting in τ^* (for c^*, K^*, τ^* of $\mathcal{A}$'s choice). This in particular requires $\mathcal{A}$ to know pk, which is freshly generated by Gen. Therefore, the probability that (even a computationally unbounded) $\mathcal{A}$ can predict pk is upper-bounded by $H_\infty(pk : (pk, \star) \leftarrow \mathsf{Gen})$. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{10,11} \leq H_\infty(pk : (pk, \star) \leftarrow \mathsf{Gen}).$$

The game above corresponds to game G_{15} in [ABJ25]. [ABJ25] does not include the condition $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, i.e., it appears that this game rewrites what our Game 2 does, which is incorrect.

The argument in this game critically relies on pk being included in H_{tag} inputs.

Game 12: Again in the case where $\neg(\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c)$, find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}^*, pk, c^*, K^*)$ query whose result is τ^*. (The difference is that Game 11 only goes over all of $\mathcal{A}$'s queries in the form of $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$, whereas this game goes over all of $\mathcal{A}$'s queries in the form of $H_{\mathsf{tag}}(\star, pk, c^*, \star)$.) If there is none or more than one, set $SK := \perp$. Otherwise there is a unique such $H_{\mathsf{tag}}(\mathsf{pw}^*, pk, c^*, K^*)$ query, and

- If $\mathsf{pw}^* \neq \mathsf{pw}$, set $SK := \perp$.
- If $\mathsf{pw}^* = \mathsf{pw}$, run the remaining part of P''s algorithm except that we now use pw^* instead of pw — that is, if there is no $\langle \mathsf{pw}^*, sk^* \rangle$ record then set $SK := \perp$; otherwise compute $K := \mathsf{Decap}(sk^*, c^*)$, and if $K^* = K$ then set $SK := H_{\mathsf{tag}}(\mathsf{pw}^*, pk, c^*, K)$, otherwise set $SK := \perp$.

If $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\star, pk, c^*, \star)$ query resulting in τ^*, then in particular $\mathcal{A}$ makes no $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^*, so both Game 11 and Game 12 set $SK := \perp$. The probability that $\mathcal{A}$ makes two different $H_{\mathsf{tag}}(\star, pk, c^*, \star)$ queries both resulting in τ^* is upper-bounded by $q_{\mathsf{tag}}(q_{\mathsf{tag}} - 1)/2^{2n+1}$. Now suppose $\mathcal{A}$ makes a unique $H_{\mathsf{tag}}(\mathsf{pw}^*, pk, c^*, K^*)$ query resulting in τ^*:

- If $\mathsf{pw}^* \neq \mathsf{pw}$, then $\mathcal{A}$ does not make a $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, \star)$ query resulting in τ^* (because there is only one $H_{\mathsf{tag}}(\star, pk, c^*, \star)$ query resulting in τ^*, and it is on pw^*), so both Game 11 and Game 12 set $SK := \perp$.
- If $\mathsf{pw}^* = \mathsf{pw}$, then there is no difference between Game 11 (which uses pw) and Game 12 (which uses pw^*).

$P(\mathsf{pw})$ | $\mathcal{A}$ | $P'(\mathsf{pw}')$

$s \leftarrow \{0,1\}^{3n}; T \leftarrow \mathbb{G}$

$\xrightarrow{s, T}$ $\xrightarrow{s^*, T^*}$

if $(s^*, T^*) = (s, T)$
$\quad SK', \tau' \leftarrow \{0,1\}^n$
$\quad (pk', \star) \leftarrow \mathsf{Gen}$
$\quad (c, \star) \leftarrow \mathsf{Encap}(pk')$
otherwise
$\quad t' := H_2(\mathsf{pw}', T^*)$
$\quad r' := s^* \oplus t'$
$\quad R' := H_1(\mathsf{pw}', r')$
$\quad pk' := T^*/R'$
$\quad (c, K') \leftarrow \mathsf{Encap}(pk')$
$\quad SK' := H_{\mathsf{key}}(\mathsf{pw}', pk', c, K')$
$\quad \tau' := H_{\mathsf{tag}}(\mathsf{pw}', pk', c, K')$

$\xleftarrow{c^*, \tau^*}$ $\xleftarrow{c, \tau'}$

if $\mathsf{pw} = \mathsf{pw}' \wedge (s^*, T^*) = (s, T) \wedge c^* = c$
$\quad$ if $\tau^* = \tau$
$\quad\quad SK := SK'$
$\quad$ otherwise
$\quad\quad SK := \perp$
otherwise
$\quad$ find $\mathcal{A}$'s $H_{\mathsf{tag}}(\mathsf{pw}, pk, c^*, K^*) = \tau^*$ query
$\quad$ if none or more than one
$\quad\quad SK := \perp$
$\quad$ fetch $\langle \mathsf{pw}, \star, sk \rangle$ record
$\quad$ if none
$\quad\quad t := H_2(\mathsf{pw}, T)$
$\quad\quad (pk, sk) \leftarrow \mathsf{Gen}$
$\quad\quad r := s \oplus t$
$\quad\quad R := T/pk$
$\quad\quad H_1(\mathsf{pw}, r) := R$
$\quad K := \mathsf{Decap}(sk, c^*)$
$\quad$ if $K^* = K$
$\quad\quad SK := H_{\mathsf{key}}(\mathsf{pw}, pk, c^*, K)$
$\quad$ otherwise
$\quad\quad SK := \perp$
output SK | | output SK'

on $\mathcal{A}$'s fresh $H_2(\mathsf{pw}^*, T) =: t^*$ query:
if P has not made this query
$\quad (pk^*, sk^*) \leftarrow \mathsf{Gen}$
$\quad r^* := s \oplus t^*$
$\quad R^* := T/pk^*$
$\quad H_1(\mathsf{pw}^*, r^*) := R^*$
$\quad$ store $\langle \mathsf{pw}^*, pk^*, sk^* \rangle$

Figure 1: Game 10 (with various aborting conditions omitted)

We conclude that

$$\mathbf{Dist}_{\mathcal{Z}}^{11,12} \leq \frac{q_{\mathsf{tag}}(q_{\mathsf{tag}}-1)}{2^{2n+1}}.$$

The game above is missing in [ABJ25]. The H_{tag} collision case was ruled out in game G_2 in [ABJ25] (as aborting condition [A1])[4]*; however, conceptually speaking there still needs to be such a bridge game, to make sure that the game hops match the ideal world at the end.*

The definition of this game, together with its argument, critically rely on pw *being included in H_{tag} inputs.*

Password extraction from P-to-P' message.

Game 13: In the case where $(s^*, T^*) \neq (s, T)$, find $\mathcal{A}$'s $H_2(\mathsf{pw}^*, T^*)$ query[5] (let the result be t^*) that satisfies *either* of the following:

1. $\mathcal{A}$ queried $H_1(\mathsf{pw}^*, r^*)$ *before* the $H_2(\mathsf{pw}^*, T^*)$ query, where $r^* = s^* \oplus t^*$.
2. $\mathcal{A}$ also queried $H_2(\mathsf{pw}^*, T)$, and

$$s \oplus s^* = H_2(\mathsf{pw}^*, T) \oplus t^*.^{6}$$

(This $H_2(\mathsf{pw}^*, T^*)$ query is what's called the "anchor query" in [JRX25].) If there is more than one such query, abort.

The aborting condition is met only in the following three cases:

- There is a collision in type (1) above, i.e., there are $r^*, t^*, (r^*)', (t^*)'$ such that

  $$r^* \oplus t^* = (r^*)' \oplus (t^*)'$$

  (so $\mathcal{A}$ can set s^* to be this value). Note that all four values in the equation are determined by two (different) H_1 queries and one H_2 query, and for each set of these queries, there is a $1/2^{3n}$ chance that the equation will hold. Therefore, the probability of this case is upper-bounded by $q_1(q_1 - 1)q_2/2^{3n}$.[7]

[4] In our game hops, we ruled out H_{tag} collisions for different K^* values in Game 2, and then additionally rule out H_{tag} collisions for different (pw^*, K^*) pairs in Game 12. We could have done both in Game 2 and avoided the additional loss here; however, we believe the current approach is clearer.

[5] The notations are slightly messed up, since the pw^* here has nothing to do with what's called pw^* in Game 12. Essentially, the pw^* here is $\mathcal{A}$'s guess of P''s password (which is called $(\mathsf{pw}')^*$ in the simulator), whereas the pw^* in Game 12 is $\mathcal{A}$'s guess of P's password. The good news is that the pw^* in Game 12 is never reused once Game 12 is done, so there is no potential confusion.

[6] Note that in this case $T^* \neq T$ since otherwise $T^* = T \Rightarrow H_2(\mathsf{pw}^*, T) = t^* \Rightarrow s^* = s \oplus H_2(\mathsf{pw}^*, T) \oplus t^* = s$, so $(s^*, T^*) = (s, T)$ which violates the condition in this game. (A very similar argument was made in the proof of Claim 1 in Game 2.)

[7] A sloppier argument would say that there are q_1 possible r^* values and q_2 possible t^* values, so there are $q_1 q_2$ possible $r^* \oplus t^*$ values and the probability of a collision is upper-bounded by $q_1 q_2(q_1 q_2 - 1)/2^{3n+1}$. This yields a looser bound since those $r^* \oplus t^*$ values are not all independent of each other. The same goes for the two cases below.

- There is a collision in type (2) above, i.e., there are $\mathsf{pw}^*, t^*, (\mathsf{pw}^*)', (t^*)'$ such that
$$H_2(\mathsf{pw}^*, T) \oplus t^* = H_2((\mathsf{pw}^*)', T) \oplus (t^*)'$$
(so $\mathcal{A}$ can set s^* to be $s \oplus H_2(\mathsf{pw}^*, T) \oplus t^*$). Note that all four values in the equation are determined by three (different) H_2 queries $H_2(\mathsf{pw}^*, T)$, $H_2((\mathsf{pw}^*)', T)$ and $H_2(\mathsf{pw}^*, T^*)$; by an argument similar to above, the probability of this case is upper-bounded by $q_2(q_2-1)(q_2-2)/2^{3n}$.

- There is a collision between type (1) and type (2), i.e., there are $r^*, t^*, (\mathsf{pw}^*)'$, $(t^*)'$ such that
$$r^* \oplus t^* = s \oplus H_2((\mathsf{pw}^*)', T) \oplus (t^*)'$$
(so $\mathcal{A}$ can set s^* to be this value). Note that s is already fixed, and all four other values in the equation are determined by two (different) H_2 queries and one H_1 query; by an argument similar to above, the probability of this case is upper-bounded by $q_1 q_2(q_2-1)/2^{3n}$.

Summing up, we have
$$\mathbf{Dist}_{\mathcal{Z}}^{12,13} \le \frac{q_2[q_1(q_1+q_2-2)+(q_2-1)(q_2-2)]}{2^{3n}}.$$

This game corresponds to part of game G_2 in [ABJ25] (as aborting conditions [A2a] and [A2b2]). The game in [ABJ25] aborts more often than necessary, though: every time $\mathcal{A}$ makes an $H_2(\star, \star)$ query, the game performs the checks and aborts accordingly, while it is only necessary to go over all of $\mathcal{A}$'s $H_2(\star, T^*)$ queries for the specific T^* as part of $\mathcal{A}$'s message to P'.

Performing the checks on $\mathcal{A}$'s H_2 query, instead of when $\mathcal{A}$ sends the (s^*, T^*) message, also adds more bookkeeping and complicates the proof. In [ABJ25], after performing the checks, the game (if it does not abort) computes the corresponding KEM public key and stores it together with (pw^*, T^*) (the record is called ForwS); later when $\mathcal{A}$ sends (s^*, T^*), the game first fetches the record to recover pw^*, and if $\mathsf{pw}^* = \mathsf{pw}'$, the game then recovers the KEM public key and uses it to complete the remaining part of P''s algorithm. This creates a conceptual discrepancy with the ideal world, and a game hop (game G_{16} in [ABJ25]) is needed. By performing the checks when $\mathcal{A}$ sends the (s^*, T^*), we avoid all these and skip their game G_{16}.

Correct password guess for P'.

Game 14: Again in the case where $(s^*, T^*) \neq (s, T)$, if pw^* is defined in type (2) in Game 13 and $\mathsf{pw}^* = \mathsf{pw}'$, fetch record $\langle \mathsf{pw}^*, pk^*, \star \rangle$ (there must be one since such a record is created when $\mathcal{A}$ queries $H_2(\mathsf{pw}^*, T)$, as per Game 5) and set $pk' := pk^* \cdot (T^*/T)$.

In Game 13, we have (in P''s algorithm)
$$pk' = \frac{T^*}{R'} = \frac{T^*}{H_1(\mathsf{pw}', r')} = \frac{T^*}{H_1(\mathsf{pw}', s^* \oplus t')} = \frac{T^*}{H_1(\mathsf{pw}', s^* \oplus H_2(\mathsf{pw}', T^*))},$$

but also (in the creation of the record $\langle \mathsf{pw}^*, pk^*, \star \rangle$)

$$pk^* = \frac{T}{R^*} = \frac{T}{H_1(\mathsf{pw}^*, r^*)} = \frac{T}{H_1(\mathsf{pw}^*, s \oplus t^*)} = \frac{T}{H_1(\mathsf{pw}^*, s \oplus H_2(\mathsf{pw}^*, T))}.$$

The condition in this game specifies that $\mathsf{pw}^* = \mathsf{pw}'$, and type (2) in Game 13 specifies that $s \oplus H_2(\mathsf{pw}^*, T) = s \oplus H_2(\mathsf{pw}^*, T^*)$. This means that the denominators in the expressions of pk' and pk^* are equal, and thus $pk' = pk^* \cdot (T^*/T)$ — exactly as in Game 14. We have

$$\mathbf{Dist}_{\mathcal{Z}}^{13,14} = 0.$$

This game corresponds to game G_{17} in [ABJ25].

Incorrect password guess for P'.

Game 15: Again in the case where $(s^*, T^*) \neq (s, T)$, if $\mathsf{pw}^* \neq \mathsf{pw}'$ or no pw^* is defined, sample $SK', \tau' \leftarrow \{0,1\}^n$. ($c$ is still computed as before.)

Let Query_3 be the event that $\mathcal{A}$ queries *either* $H_{\mathsf{key}}(\mathsf{pw}', pk', c, K')$ *or* $H_{\mathsf{tag}}(\mathsf{pw}', pk', c, K')$. Clearly Game 14 and Game 15 are identical unless Query_3 happens.

References

[ABJ25] Afonso Arriaga, Manuel Barbosa, and Stanislaw Jarecki. NoIC: PAKE from KEM without ideal ciphers. Cryptology ePrint Archive, Report 2025/231, 2025.

[BS23] Dan Boneh and Victor Shoup. A graduate course in applied cryptography (version 0.6), 2023. https://toc.cryptobook.us/book.pdf.

[JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In *EUROCRYPT 2025*, pages 451–481, 2025.

[MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In *CCS 2020*, pages 425–442, 2020.